

~~Spring in Action~~

(Spring → Spring application context, container 제공.)

↓
컴포넌트 생성/관리

Bean의 상호연결 → 'Dependency 주입'

① Configuration ⇒ 각 빈을 스프링 애플리케이션 컨텍스트에 제공하는 구성 클래스

```
public class ServiceConfiguration {
```

② Bean ⇒ 구성 클래스의 method.

```
    public InventoryService inventoryService() {  
        return new InventoryService();  
    }
```

③ Bean

```
    public ProductService productService() {  
        return new ProductService(inventoryService());  
    }  
}
```

1) 스프링 프로젝트 설정

- Name : 이름
- Location : 프로젝트 생성 경로
- Language : Java, kotlin, groovy
- Type : gradle / Maven
- Group : 프로젝트 그룹
- Artifact : 프로젝트 식별자
- packagename : 패키지 이름 (클래스나 파일이 속하는 패키지) ... 소스코드 상단에 선언
- JDK : JDK 버전
- Java : Java 버전은 설정한 JDK와 동일한 버전으로 선택
- Packaging : Spring boot는 내장서버 지원
Jar2 설정하면 application 독립 실행 o.
배포 / 유지보수 간소화

• 의존성

- Spring Web : 웹 개발 핵심 의존성. (웹 요청 처리 / MVC 패턴 / 템플릿 / JSON)
- Spring boot DevTools : 개발 생산성 ↑ (자동 애플리케이션 재시작, 브라우저 자동 새로고침, 캐시 바깥 ...)
- Lombok : Java 반복적인 부분 줄여줌 (getter/setter 등 생성. ex) @Getter, ToString...
- Thymeleaf : 서버 사이드 템플릿 엔진 (html 문법 서버 사이드 렌더링 ...)

• 파일 구조

소스 코드 => src/main/java 테스트 코드 => src/test/java

Java 리소스 X => src/main/resources

mvnw/mvnw.cmd => 메아본 러퍼 스크립트.

maven과 복사 PC에 설치 X더라도 이 스크립트 이용해서 빌드.

pom.xml => maven 빌드 명세.

TacoCloudApplication.java

=> Spring Boot Main Class.

applications.properties : 구성 속성 지정 가능

Static : 브라우저에 제공할 정적인 콘텐츠 (image, stylesheet, js ...)

templates : 브라우저에 콘텐츠 보여주는 템플릿 파일. (ex) thymeleaf ...)

TacoCloudApplicationTests.java => 스프링 애플리케이션이 성공적으로 로드 되는지 확인하는 간단한 test class.

pom.xml

<parent> 요소의 <version>.

: 부모 POM으로 spring boot starter parent 있다는 것 지정.

↳ 여러 라이브러리의 의존성 관리 제공.

<dependencies> => 의존성 관리 요소.

<dependency> 요소로 지정

<artifactId> 가 starter spring boot starter 의존성

- 애플리케이션의 부트 스트랩 (구동)

package tacos;

import org.springframework

import org.springframework

@SpringBootApplication ← spring boot application (스프링 부트 애플리케이션으로 명시)

public class TacoCloudApplication {

public static void main(String[] args) {

SpringApplication.run(TacoCloudApplication.class, args);

← 애플리케이션 실행

}

}

@SpringBootConfiguration

↓
결합

@SpringBootConfiguration

: 구성 클래스로 지정

@EnableAutoConfiguration

: 스프링 부트 자동-구성 활성화

@ComponentScan

: 컴포넌트 검색 활성화

↳ @Component, @Controller, @Service...

- main()

=> JAR 파일 실행될 때 호출되어 실행되는 메소드

대부분 표준화된 형식의 코드로 구성.

애플리케이션 시작 —> run() 호출 —> run() 메소드에 매개변수 전달

Spring application context 생성하는
SpringApplication 클래스의 메소드

구성클래스, 명령행 인자.

2) 테스트 클래스

- 1개의 Test Method. , 실행코드 X

- 실증적으로 코드될 수 있는지 확인하는 기본적인 검사.

③ SpringBootTest => 스프링 부트 기능으로 테스트 시작한다고 JUnit 이 알려줌.

3) 애플리케이션 작성.

- 홈페이지.

- 웹 요청 처리하는 controller 클래스
- 요청 정제하는 뉴 템플릿.

Model View Controller

- 웹 요청 처리하기.

Controller : 웹 요청과 응답 처리하는 컴포넌트.

: 실행적으로 모델 데이터 캐처서 응답.

: 브라우저에 반환되는 html 생성하여 해당 응답의 웹 요청

View에 전달.

src/main/java 에서 HomeController 클래스 생성.

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
```

@Controller ← 컨트롤러.

```
public class HomeController {
```

@GetMapping("/") ← 루트 경로인 / 이 웹 요청 처리

```
public String home() {
```

```
    return "home"; ← View 이름 반환.
```

```
}
```

```
}
```

⇒ Controller 클래스가 컴포넌트 식별

복사

@Controller 동일 기능 ⇒ @Component, @Service, @Repository

home() ⇒ 루트 경로 / 이 http GET 요청이 수신되면
home()이 해당 요청 처리.

- View 정하기

src/main/resources/templates

```
<!DOCTYPE html>
```

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns="http://www.thymeleaf.org">
```

```
<head>
```

```
:
```

```
</head>
```

```
<body>
```

```
<h1> Test </h1>
```

```

```

```
: thymeleaf-1 th:src 속성 :
```

```
< /src/main/resources/static/images
```

- 컨트롤러 테스트

루트 경로인 / 이 http GET 요청 수행 후 성공인거, view 이름이 home 인

This is home.html 메시지 나오는지 테스트

① /src/test/java 아래 tocos 패키지

② New => Junit Test Case 선택
있으면 원하는 클래스 선택

import 문 생략

④ WebMvcTest (HomeController.java) ← Spring boot 제공. 스프링 MVC 애플리케이션 형태로 테스트 실행되도록 한다.

실제 서버에선 스프링 MVC가 많이 매커니즘 사용.
테스트 클래스에 MockMvc 객체 주입.

④ Autowired
private MockMvc mockMvc;

④ Test

public void testHomePage() throws Exception {

mockMvc.perform(get("/")) → GET / 를 수행.

.andExpect(status().isOk()) : HTTP 200 되어야한다.

.andExpect(view().name("home")) ← home View 있어야함.

.andExpect(content().string(containsString(" ~ ")));

이러지
기대하는 것

: 컨트롤러가 전송하는 문자가 포함.

testHomePage() : 홈페이지에 대해 수행하고자 하는 테스트 장.

① 루트 경로인 / 이 HTTP GET 요청을 MockMvc 객체로 수행.

② 2가지 기대하는 것

- 응답은 반드시 HTTP 200
- View 이름 : home
- 원하는 텍스트 있어야한다

하나라도 실패하면 Test는 실패.

빌드 및 실행.

localhost: 8080 / 하면 home.html 내용 나옴.

통캣 설치 안했는데 실행 가능한 이유

=> Spring boot application 이 실행에 필요한 모든 것이 포함.

Tomcat 이 우리 애플리케이션의 일부임.

- Spring boot DevTools.

: 스프링 개발자에게 편리한 도구 제공

- 코드 변경 시 자동으로 애플리케이션 다시 시작.
- 브라우저로 전송되는 리소스가 변경되면 자동 새로고침.

⋮

JS, CSS, template

- 템플릿 캐시 자동 비활성화
- DB가 H2라면 자동으로 H2 콘솔 활성화 (사용중)

(단): - 설정 변경 적용 X => 클래스 코드는 자동으로 다시 로드 X.

: 자동으로 브라우저 새로고침 / 템플릿 캐시 비활성화

L> Thymeleaf / FreeMarker 같은 템플릿에서는

템플릿이 화상 결과 캐시에 저장되고 사용.

=(코드 분석)

실제 운영에는 좋지않은 개발 습관이므로 사용 X.

Spring 목표 => Web application 생성, DB 사용, 보안, micro service 등에 노력됨.

=> 의존성 관리, 자동구성 등 내버려둬도 되는 기능.